Milestone 1 – notatka dla zespołu

Wykrywanie spamu w SMS-ach: co jest na slajdach, co znaczą trudne słowa i o co mogą zapytać

Jerzy Czarnecki, Mateusz Małachowski, Sebastian Rydz, Wojciech Wójcik · 29.10.2026

1. O co chodzi w projekcie (w dwóch zdaniach)

Mamy ok. 5,6 tys. SMS-ów, każdy oznaczony przez człowieka jako **spam** albo **ham**. Budujemy program, który sam zgaduje, czy SMS to spam, i sprawdzamy, **czy przerobienie tekstu przed nauką** (np. zamiana numerów telefonów na wspólny znacznik) **poprawia wynik**. Milestone 1 jest *przed* jakimkolwiek modelem: obejrzeliśmy dane, podzieliliśmy je i postawiliśmy hipotezy.

Podział na trzy części – to trzeba rozumieć, żeby cokolwiek dalej miało sens:

- **train (70%)** – na tym model się uczy,
- **val / walidacja (15%)** – próbny sprawdzian; tu wybieramy ustawienia,
- **test (15%)** – prawdziwy egzamin, używany **raz, na końcu**. Nie wolno na nim niczego dobierać.

Przeciek (leakage): gdy ten sam (albo prawie ten sam) SMS jest i w treningu, i w teście – model „zna odpowiedź na pamięć" i wynik wychodzi lepszy niż w rzeczywistości.

2. Slajd po slajdzie

Slajd 1 – tytułowy

Tytuł, skład, data. Nic do tłumaczenia.

Slajd 2 – „Dane: 5574 SMS-y, 13,4% spamu"

SMS Spam Collection v.1, UCI #228, CC BY 4.0

Nazwa zbioru danych: jeden plik tekstowy, każda linia to etykieta + treść SMS-a, zebrany w 2011 r. **UCI** = UCI Machine Learning Repository (Uniwersytet Kalifornijski w Irvine) – publiczna „biblioteka" zbiorów danych; **228** to numer zbioru w tej bibliotece. **CC BY 4.0** to licencja: wolno używać, trzeba podać autorów.

ham / spam

spam – niechciana wiadomość (reklama, „wygrałeś nagrodę, zadzwoń pod 0906…"). **ham** – normalna wiadomość od człowieka („będę za 10 min"). Nazwa to żart: spam to marka mielonki, więc „dobra" wiadomość to szynka (ham). 4827 ham i 747 spam – spamu jest tylko 13,4%, czyli klasy są **niezbalansowane**.

- **Braki i puste: 0** – żaden SMS nie ma pustej treści ani brakującej etykiety.
- **Emoji: 0 (rok 2011)** – ówczesne telefony praktycznie ich nie wysyłały, więc eksperyment z emoji nie ma sensu.
- `id` **= numer linii pliku** – każdy SMS ma numer (linia w oryginalnym pliku, od 0). Prowadzący wymaga zapisania identyfikatorów, żeby było wiadomo, który SMS trafił do testu.

35 SMS-ów z błędnym cp1252

Komputer nie przechowuje liter, tylko liczby. **Kodowanie znaków** to tabelka „liczba → litera". **UTF-8** – współczesny standard, w nim jest plik. **cp1252** (Windows-1252) – stara tabelka Windowsa, w której liczba 0x92 to ładny apostrof ’. W 35 SMS-ach nikt tego nie przekonwertował i zamiast apostrofu siedzi niewidoczny znak sterujący. Ciekawostka o jakości danych, nie poważny problem.

Metryka główna: F1 dla klasy spam

Dlaczego nie zwykłe „% trafień" (accuracy)? Bo „model", który *zawsze* mówi „ham", ma 86,6% trafień i nie łapie ani jednego spamu.

- **precision (precyzja)** – z tego, co model nazwał spamem, ile naprawdę było spamem. Niska = SMS-y od mamy lądują w spamie.
- **recall (czułość)** – z całego prawdziwego spamu, ile model złapał. Niski = spam przecieka.
- **F1** – średnia tych dwóch, która „karze", gdy któraś jest niska. Od 0 do 1, im więcej, tym lepiej.

Wykres długości. Oś X – długość SMS-a w znakach, oś Y – jaki % wiadomości ma taką długość; niebieski = ham, pomarańczowy = spam. Kreska na 160 znakach to **limit jednego SMS-a**. Spamerzy upychają treść pod limit (**mediana** 149 znaków, czyli połowa krótsza, połowa dłuższa), ludzie piszą krótko (mediana 53).

Slajd 3 – „Duplikaty i podział bez przecieku"

- **403 dokładne duplikaty (7,2%)** – kopie słowo w słowo. Spam powtarza się 2× częściej, bo spamer wysyła to samo do wielu ludzi.
- **0 tekstów z konfliktem etykiet** – nie ma tekstu oznaczonego raz jako spam, a raz jako ham.

Bliskie duplikaty – trudniejszy problem: „URGENT! You won £800, call 0906…1234" i „…£900, call 0906…9876" to ten sam szablon, choć nie słowo w słowo. Wykrywamy je tak:

n-gramy znakowe 3–5

Kroimy tekst na kawałki po 3, 4 i 5 kolejnych liter. `call` → `␣ca`, `cal`, `all`, `ll␣`, … (spacje na brzegach dokleja ustawienie `char_wb` – „w obrębie słowa"). Dwa SMS-y z tego samego szablonu mają prawie te same kawałki, nawet gdy różnią się numerem czy literówką.

TF-IDF – zamiana tekstu na liczby

Każdy możliwy kawałek (słowo albo n-gram) to kolumna tabeli, każdy SMS to wiersz liczb. **TF** (term frequency) – ile razy kawałek jest w tym SMS-ie. **IDF** (inverse document frequency) – kara za popularność: kawałek obecny prawie wszędzie dostaje małą wagę, rzadki – dużą. **TF×IDF** = ważne jest to, co jest w tym tekście, a nie jest wszędzie.

Cosinus (podobieństwo kosinusowe)

Każdy SMS to teraz lista liczb, czyli strzałka. Cosinus kąta między dwiema strzałkami mówi, jak bardzo wskazują w tę samą stronę: **1** = prawie identyczne, **0** = nic wspólnego. Nie zależy od długości tekstu, tylko od kierunku. Przyjęliśmy **próg 0,7**: poniżej zaczynały się sklejać różne krótkie hamy („I'm home." i „I'm at home. Please call").

- **4897 grup (największa: 40)** – duplikaty łączymy łańcuchowo: A podobne do B, B do C ⇒ A, B, C w jednej grupie. Większość grup to pojedyncze SMS-y. Największa (40) to automatyczny ham „Sorry, I'll call later".

- **Decyzja** – nie wyrzucamy duplikatów (w prawdziwym świecie spam też przychodzi wiele razy), ale **cała grupa trafia do jednej części** (train albo val albo test). Dzięki temu nie ma przecieku.
- **Tabela** – % spamu prawie identyczny w każdej części (13,4 / 13,4 / 13,5). To jest **stratyfikacja**: każda część ma takie proporcje klas jak całość.

70/15/15, StratifiedGroupKFold, seed 42

StratifiedGroupKFold – gotowe narzędzie ze scikit-learn: tnie dane na 20 kawałków, pilnując naraz proporcji klas (*Stratified*) i tego, żeby grupa się nie rozdzieliła (*Group*). 14 kawałków → train, 3 → val, 3 → test. **seed 42** – „ziarno" losowania; to samo ziarno = zawsze to samo losowanie, więc każdy odtworzy identyczny podział. Prowadzący narzucił 42.

Jeśli zapytają „skąd wiecie, że nie ma przecieku?" – sprawdziliśmy po fakcie, że **żaden** SMS z walidacji ani testu nie ma w treningu odpowiednika z podobieństwem $\geq 0{,}7$. Tego nie ma już na slajdzie, ale to jest odpowiedź.

Slajd 4 – „Co odróżnia spam od hamu (zbiór treningowy)"

„Zbiór treningowy" w tytule jest ważny: statystyki liczyliśmy **tylko na train**, żeby test nie wpływał na nasze pomysły. Wykres pokazuje, jaki % hamów i spamów ma daną cechę:

- **Numer telefonu ($\geq$ 8 cyfr)** – 61,5% spamu, 0,1% hamu. Prawie idealny sygnał.
- **Kwota (£, $, 150p)** – „wygraj £1000", „150p za wiadomość" (p = pensy).
- **Kod krótki (5–6 cyfr)** – numery premium typu „wyślij STOP na 87121".
- **URL / domena** – linki do stron.
- **Encja HTML (`< >`)** – tak w HTML-u zapisuje się znaki < i >. Tylko w hamie (wyjaśnienie niżej).
- **Wielokropek, `!!!`, emotikony `:)`** – częściej w hamie, bo tak piszą ludzie.

Rzeczy, które były w ramkach obok wykresu – **warto je powiedzieć na głos**:

- **Numery są silne, ale nowe**: w 83,6% spamów z walidacji z numerem telefonu żaden numer nie pojawił się w treningu. Model uczy się konkretnych słów, więc numer widziany raz nic mu nie da przy nowym numerze. To uzasadnia P1.
- **Artefakt** `lt`/`gt`: większość hamów pochodzi z korpusu NUS (National University of Singapore). Jego autorzy zastąpili liczby napisem `<#>`. Model widzi z tego słowa `lt` i `gt` i uczy się „`lt` = ham". To nie cecha języka, tylko ślad po źródle danych (ukryty brak danych).
- **Interpunkcja to ham, ale B1 jej nie widzi** – patrz `token_pattern` na slajdzie 6.

Slajd 5 – „Trzy hipotezy"

Zasada z regulaminu: **model się nie zmienia, zmienia się tylko tekst na wejściu**. Tylko wtedy wiadomo, że różnica w wyniku wynika z przetwarzania. **Hipoteza** = przewidywanie zapisane *przed* eksperymentem, z uzasadnieniem.

B1 – model bazowy

Surowy tekst → TF-IDF na słowach → **regresja logistyczna**. Mimo nazwy to klasyfikator: każde słowo dostaje wagę („prize" mocno na spam, „lol" na ham), wagi się sumują i wychodzi prawdopodobieństwo spamu 0–1. P1–P3 to dokładnie B1, tylko z przerobionym tekstem.

P1 – maskowanie numerów i kwot. Maskowanie = zamiana konkretnej wartości na wspólny znacznik: `call 09061234567` → `call <PHONE>`, `£800` → `<MONEY>`. *Dlaczego:* w **BoW** (bag of words, „worek słów" – tabela z TF-IDF, każde słowo to kolumna) każdy numer jest osobną kolumną, która występuje w 1–2 SMS-ach, więc model nie ma się z czego nauczyć. Po maskowaniu wszystkie numery to jedno słowo `<PHONE>` obecne w setkach spamów. *Oczekujemy:* wyższego recall i F1. *Ryzyko:* ham „oddam ci $700" dostanie `<MONEY>` → fałszywy alarm.

P2 – URL: maskowanie vs usuwanie. Albo `<URL>`, albo wycięcie linku. *Dlaczego mały efekt:* tokenizer już teraz tnie `www.getzed.co.uk` na `www`, `getzed`, `co`, `uk`, a samo `uk` jest w 50 spamach i 1 hamie. *Oczekujemy:* maskowanie ≈ bez zmian, usunięcie lekko obniży recall. Wariant mierzy, ile model zawdzięcza linkom.

P3 – normalizacja powtórzeń. `sooo` → `soo`, `!!!!` → `!!`. *Oczekujemy:* braku istotnej zmiany – powtórzenia są rzadkie i podobnie częste w obu klasach. Ewentualny zysk w **teście odporności na literówki** (celowo psujemy tekst testowy i patrzymy, jak spada wynik – temat tego wymaga). Brak efektu jest OK: regulamin mówi wprost, że nie musi być poprawy po każdej operacji.

Najpierw każdy wariant osobno, potem połączenie najlepszych (np. P1 + P2).

Slajd 6 – „Problemy i plan na Milestone 2"

Problemy:

- **Mały test: 113 spamów, 1 SMS ≈ 0,9 p.p. recall** – jeden złapany/przepuszczony spam zmienia recall o $1/113 \approx 0{,}9$ **punktu procentowego**. Różnice między wariantami mogą być przypadkiem.
- **65 ze 113 spamów testu jest w grupach duplikatów** – te kampanie liczą się w teście kilka razy, więc pokażemy też wynik na teście bez kopii (781 unikalnych SMS-ów z 837).
- **41 spamów testu przypomina kampanie z treningu (cos 0,5–0,7)** – nie duplikaty, ale podobne; mogą zawyżać wynik, więc pokażemy też wynik tylko na SMS-ach naprawdę nowych (cos < 0,5).

Sparowany bootstrap

Sposób sprawdzenia, czy różnica jest prawdziwa. Losujemy z testu (ze zwracaniem – ten sam SMS może wypaść kilka razy) nowy „test" tej samej wielkości i liczymy wynik B1 i wariantu P **na tych samych wylosowanych SMS-ach** (stąd „sparowany"). Powtarzamy ≥ 1000 razy i dostajemy **przedział ufności** różnicy, np. „P1 lepszy o 2–5 p.p.". Jeśli przedział obejmuje zero – nie możemy powiedzieć, że wariant pomaga. Prowadzący tego wymaga.

Plan M2:

- **B0** – najgłupszy „model": zawsze mówi ham. F1 spamu = 0. Punkt zero.
- **B1** – jak wyżej. Ustawienia muszą być jawne:

lowercase, token_pattern, C, próg decyzji

`lowercase` – wszystko na małe litery („FREE" i „free" to to samo słowo).
`token_pattern` = `\b\w\w+\b` – reguła cięcia tekstu na słowa: **token to ciąg co najmniej 2 liter lub cyfr**, cała reszta wypada. „Call me!!! :)" → `call`, `me` (wykrzykniki i `:)` znikają); „I am 5 min late" → `am`, `min`, `late` (`I` i `5` – tylko 1 znak). **Uwaga:** to *nie* jest zamiana `!!!` na `!!` – to P3. `token_pattern` to domyślne zachowanie modelu bazowego, nie nasze przetwarzanie.
C – siła **regularyzacji** regresji logistycznej. Za mała regularyzacja → model uczy się szczegółów na pamięć (overfitting); za duża → zbyt ogólny. **Pułapka:** w scikit-learn **większe C = słabsza regularyzacja** (C to odwrotność siły).
Próg decyzji – od jakiego prawdopodobieństwa mówimy „spam". Domyślnie 0,5; np. 0,3 = więcej złapanego spamu, ale więcej fałszywych alarmów. **C i próg dobieramy tylko na walidacji**, potem

zamrażamy dla P1–P3, żeby porównanie było uczciwe.

- **P1–P3 na zamrożonym B1**, potem połączenie najlepszych.
- **Osobny eksperyment: n-gramy słowne vs znakowe** – opis poniżej.

n-gramy słowne vs znakowe

Chodzi o to, na jakie „kawałki" tniemy tekst przed TF-IDF.
Słowne – kawałek = całe słowo (lub para słów). „free entry now" → `free`, `entry`, `now` (+ dla 2-gramów: `free entry`, `entry now`). *Plus:* czytelne. *Minus:* każda literówka to dla modelu zupełnie nowe słowo – `free`, `fr33`, `freee` to trzy różne kolumny.
Znakowe – kawałek = 3–5 kolejnych liter. `freee` ma prawie te same kawałki co `free`, więc model je rozpoznaje. *Plus:* odporne na literówki i spamerskie sztuczki („fr€e"), łapie fragmenty jak `0906` z numerów premium. *Minus:* dużo więcej kolumn, mniej czytelne.
Dlaczego osobny eksperyment, a nie wariant P? Regulamin: P1–P3 muszą mieć *tę samą reprezentację*. Zmiana słowa → znaki to zmiana reprezentacji, nie przetwarzanie tekstu. Spodziewamy się, że znakowe wypadną lepiej w teście literówek – ale to trzeba sprawdzić, nie zakładać.

3. Niewygodne pytania, które mogą paść (i co odpowiedzieć)

Tu mamy słabe miejsca – lepiej powiedzieć to samemu

P: Spam i ham pochodzą z różnych źródeł. Skąd wiecie, że model uczy się spamu, a nie źródła?

Według readme zbioru: spam – 425 z brytyjskiego forum Grumbletext + 322 z SMS Spam Corpus v.0.1; ham – 3375 z korpusu NUS (Singapur) + 450 z pracy doktorskiej C. Tag + 1002 z SMS Spam Corpus. Model może się uczyć „singapurski angielski = ham". **O:** „Tak, to zakłócenie przez źródło. `lt/gt` jest tego objawem. W analizie błędów sprawdzimy, czy model myli się na hamie spoza NUS. Zbiór nie ma kolumny ze źródłem, więc w pełni się tego nie usunie – wpiszemy to do ograniczeń."

P: Temat każe porównać koszty fałszywych alarmów i niewykrytego spamu. Gdzie to jest?

Na slajdach tego nie ma. **O:** „W filtrze SMS fałszywy alarm jest droższy – zgubiona wiadomość od człowieka boli bardziej niż jeden spam. W M2 pokażemy macierz pomyłek i jak przy różnych progach zmienia się liczba fałszywych alarmów i przepuszczonego spamu."

P: Wzorce (regexy) na telefon, kwotę, kod krótki pisaliście, patrząc na które dane? Oglądaliście test?

Do ustalenia w zespole przed prezentacją. Jeśli tylko train: „Budowaliśmy je na treningu, test nie był oglądany." Jeśli nie: przyznać się i powiedzieć, że reguły zostają zamrożone przed jakimkolwiek wynikiem na teście.

Podział i przeciek

P: TF-IDF do wykrywania duplikatów dopasowaliście na wszystkich danych, z testem. Regulamin mówi: tylko na treningu.

O: „To dotyczy przekształceń będących częścią modelu. Ten TF-IDF służy tylko do kontroli podziału, która według regulaminu jest wspólna dla wszystkich wariantów. Nie używa etykiet i nie trafia do modelu. Żeby wykryć duplikat między train a test, trzeba widzieć oba zbiory."

P: Skąd próg 0,7?

O: „Przejrzeliśmy pary przy różnych progach. Poniżej 0,7 sklejają się różne krótkie hamy, powyżej zostają szablony spamu. Wybór nie zależał od wyniku żadnego modelu." Liczby na zapas: rekordów testu z podobieństwem do treningu $\geq 0{,}6$ jest 49, $\geq 0{,}5$ – 120.

P: Czemu nie usunęliście duplikatów?

O: „Temat każe zachować naturalne proporcje w teście, a spam w praktyce przychodzi wielokrotnie. Grupowanie usuwa przeciek, a zachowanie kopii – rozkład. Raportujemy też wynik po deduplikacji (781 unikalnych)."

P: Grupy łączycie łańcuchowo. Nie powstała jedna wielka grupa z różnych tekstów?

O: „Największa ma 40 rekordów i to ten sam automatyczny ham. Żadnej pary ham–spam w jednej grupie."

P: 83,6% liczyliście na walidacji – czy to nie używanie walidacji do hipotez?

O: „Walidacja jest właśnie od podejmowania decyzji. Zakaz dotyczy testu, a test nie wpłynął na żadną hipotezę."

Metryki i statystyka

P: W walidacji jest tylko 112 spamów. Dobierając C i próg, nie przeuczycie się na walidację?

O: „Ryzyko jest. Dlatego wybieramy z niedużej siatki wartości i zamrażamy próg po B1, zamiast stroić go osobno dla każdego P. Porównanie jest wtedy uczciwe."

P: Co, jeśli bootstrap pokaże przedział obejmujący zero?

O: „Wtedy uczciwie mówimy, że wariant nie daje istotnej poprawy. Regulamin to dopuszcza."

P: Czemu F1, a nie PR-AUC jako główna?

O: „F1 spamu narzuca temat. PR-AUC raportujemy dodatkowo, bo nie zależy od progu."

Hipotezy

P: Po co P3, skoro przewidujecie brak efektu?

O: „Temat go sugeruje, brak efektu też jest wynikiem i trzeba go rzetelnie wykazać. Ma też sens w teście literówek."

P: P1 zamieni „$700" w hamie na `<MONEY>`. Nie wzrosną fałszywe alarmy?

O: „To ryzyko jest na slajdzie. Kwota jest w 0,3% hamu, więc to kilka wiadomości. Sprawdzimy precyzję i pokażemy te przypadki w analizie błędów."

P: Czy maskowanie nie jest przeciekiem etykiety? Numer telefonu prawie zawsze oznacza spam.

O: „Nie. Maskowanie to reguła na samym tekście, bez etykiet, identyczna dla train i test. To, że numer jest silną cechą, to prawdziwa własność spamu."

P: `lowercase` w B1 to już przetwarzanie. Czemu jest w modelu bazowym?

O: „To domyślne ustawienie `TfidfVectorizer`. Regulamin wymaga, żeby ustawienia normalizacji były jawne, a nie zerowe."

Ogólne – każdy musi umieć odpowiedzieć sam

P: Zbiór jest z 2011 r. Co to mówi o współczesnym spamie?

O: „Niewiele: brytyjskie numery premium, zero emoji, mało linków. Wnioski dotyczą tego korpusu – wpiszemy to do ograniczeń."

P: LLM mógł widzieć ten zbiór w treningu.

O: „Tak, jest publiczny od 2011 r. (UCI, Kaggle). Omówimy to w M4."

Definicje do przećwiczenia na głos (regulamin przewiduje indywidualne pytania): co to IDF; co mierzy cosinus i czemu nie zależy od długości tekstu; co to stratyfikacja; co robi seed; czym walidacja różni się od testu; co to precision, recall, F1; co robi C.

4. Do zrobienia przed prezentacją

1. Ustalić, **na jakich danych powstały regexy** w `patterns.py`.
2. Rozważyć dopisanie na slajd 6: **różne źródła spamu i hamu** (ryzyko), **koszty fałszywych alarmów** i **test literówek** (plan M2).
3. Przećwiczyć na głos definicje z sekcji wyżej – każdy z nas.

Notatkę przygotowano z pomocą Claude Code (Anthropic) na podstawie repozytorium projektu; liczby pochodzą z `reports/milestone1/milestone1.md` i `data/splits/split_seed42.meta.json`.